

NOTIFY

A Virtual Learning Companion

A PROJECT REPORT

Submitted by:

Tanish Bhole (221B407)

Under the Guidance of:

Dr. P. S. Banerjee

Assistant Professor, Department of Computer Science & Engineering



May - 2026

Submitted in partial fulfilment of the Degree of

BACHELOR OF TECHNOLOGY **IN** **COMPUTER SCIENCE & ENGINEERING**

Department of Computer Science & Engineering
JAYPEE UNIVERSITY OF ENGINEERING & TECHNOLOGY,
A-B ROAD, RAGHOGARH, DT. GUNA - 473226, M.P., INDIA

DECLARATION

I hereby declare that the work reported in the B.Tech project entitled **“NOTIFY – A Virtual Learning Companion”** submitted at Jaypee University of Engineering & Technology, Guna (M.P.), is an authentic record of my work carried out under the supervision of Dr. P. S. Banerjee. I have not submitted this work elsewhere for any other degree or diploma. I am fully responsible for the contents of my project report.

To the best of my knowledge, this work does not infringe any intellectual property rights of any individual or organisation. All external sources used in this work have been duly acknowledged in the References section of this report.

Tanish Bhole (221B407)

Department of Computer Science & Engineering

Jaypee University of Engineering & Technology, Guna (M.P.)

Date: _____



JAYPEE UNIVERSITY OF ENGINEERING & TECHNOLOGY, GUNA

Grade 'A+' Accredited with by NAAC & Approved U/S 2(f)

of the UGC Act, 1956 A.B. Road,
Raghogarh, Dist.: Guna (M.P.) India,

Pin-473226 Phone: 07544267051,
267310-14, Fax: 07544 267011

Website: www.juet.ac.in

CERTIFIC ATE

This is to certify that the project work entitled **“NOTIFY – A Virtual Learning Companion”** submitted by Tanish Bhole (221B407) to Jaypee University of Engineering & Technology, Guna (M.P.) in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science & Engineering, is a bona fide record of the work carried out by the student under my supervision.

The work presented in this report is original and has not been submitted to this or any other University/Institution for the award of any degree or diploma. The candidate has met all the requirements of the Major Project Part-2 for the B.Tech VIII semester and is recommended for evaluation.

Dr. P. S. Banerjee

Assistant Professor

Department of Computer Science & Engineering

Jaypee University of Engineering & Technology, Guna (M.P.)

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor Dr. P. S. Banerjee, Assistant Professor, Department of Computer Science & Engineering, Jaypee University of Engineering & Technology, Guna, for his constant guidance, encouragement and patience throughout the course of this project. His insights helped shape the architecture and the scope of Notify at every weekly review.

I would also like to thank the Head of the Department and the faculty of the Department of Computer Science & Engineering at JUET, Guna, for providing the infrastructure and academic environment that made this project possible. I am grateful to my classmates for the discussions that sharpened the requirements during the early weeks, and to the open-source authors of FastAPI, Next.js, MongoDB, Redux Toolkit and the Groq inference platform whose work made the system practical to build.

Finally, I wish to thank my family for their continuous support and patience during the twelve weeks of this project.

Tanish Bhole (221B407)

ABSTRACT

Notify is a web-based virtual learning companion that combines rich-text note-taking, AI-generated mind maps, AI-generated flashcards, a notes-aware chatbot and an in-app research browser into a single workspace. The system is built as a Next.js single-page application backed by a FastAPI service, a MongoDB database and the Groq inference platform running Llama 3.1. The design and implementation were carried out over twelve weeks as the VIII-semester Major Project Part-2 submission for the B.Tech programme at Jaypee University of Engineering & Technology, Guna.

This report documents the motivation, the literature survey of existing note, flashcard and mind-mapping tools, the system architecture, the data model, the implementation of each backend route and frontend component, the testing strategy and the production deployment to Vercel and Render with MongoDB Atlas. It concludes with the lessons learnt and a list of concrete future extensions. The deployed application demonstrates that a single student can ship a non-trivial AI-augmented full-stack application end to end inside a one-semester schedule.

Keywords: Note-taking, Flashcards, Mind Map, Large Language Model, Llama 3.1, FastAPI, Next.js, MongoDB, Redux, Virtual Learning Companion.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig. 1.1	Twelve-Week Project Timeline	5
Fig. 2.1	Comparison of Existing Note, Flashcard and Mind-Map Tools	10
Fig. 3.1	High-Level System Architecture of Notify	12
Fig. 3.2	Data Flow Diagram (Level-1) of Notify	13
Fig. 3.3	Entity-Relationship Diagram of MongoDB Collections	16
Fig. 3.4	Use-Case Diagram for Notify	17
Fig. 3.5	Module Decomposition of Notify	19
Fig. 3.6	Sequence Diagram for Mind-Map Generation	21
Fig. 3.7	Sequence Diagram for User Authentication	23
Fig. 4.1	Frontend Component Tree	27
Fig. 5.1	Postman Test Matrix	33
Fig. 5.2	End-to-End Test Results	34
Fig. 5.3	Median Endpoint Latency	35

LIST OF SYMBOLS AND ACRONYMS

Acronym	Full Form
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JUET	Jaypee University of Engineering & Technology
JWT	JSON Web Token
LLM	Large Language Model
MIT	Massachusetts Institute of Technology
MVP	Minimum Viable Product
NoSQL	Not Only Structured Query Language
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single-Page Application
SSR	Server-Side Rendering
URL	Uniform Resource Locator
UX	User Experience
YAML	YAML Ain't Markup Language

TABLE OF CONTENTS

Declaration	ii
Certificate	iii
Acknowledgement	iv
Abstract	v
List of Figures	vi
List of Symbols and Acronyms	vii
Table of Contents	viii
Chapter 1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope and Limitations	2
1.6 Project Plan and Timeline	3
1.7 Organisation of the Report	4
Chapter 2 Literature Review	5
2.1 Overview	5
2.2 Rich-Text Note-Taking Tools	5
2.3 Spaced-Repetition Flashcard Tools	5
2.4 Mind-Mapping Tools	6
2.5 AI-Augmented Study Assistants	6
2.6 Technology Building Blocks	6
2.7 Position of the Present Work	7
Chapter 3 System Analysis and Design	8
3.1 System Architecture	8
3.2 Functional Requirements	9
3.3 Non-Functional Requirements	10
3.4 Data Model	10
3.5 Use Cases	11
3.6 Module Decomposition	13
3.7 Design Decisions	13
3.8 Security Considerations	15
3.9 Performance and Scalability Design	16
Chapter 4 Implementation	17
4.1 Backend Bootstrap	17
4.2 Database Access	18
4.3 Authentication	18

4.4 Notes Save	18
4.5 Mind-Map Generation	19
4.6 Flashcard Generation	20
4.7 Chatbot	20
4.8 In-App Browser Proxy	20
4.9 Frontend Composition	21
4.10 State Management	22
4.11 Editor and Auto-Save	22
4.12 Mind-Map Rendering	22
4.13 Flashcard Carousel	22
4.14 Deployment	23
4.15 Logging and Monitoring	24
4.16 Continuous Integration and Code Quality	24
Chapter 5 Testing and Results	25
5.1 Testing Strategy	25
5.2 API Tests in Postman	25
5.3 End-to-End Manual Tests	26
5.4 Result Summary	26
5.5 Performance Measurements	27
5.6 Deployment Smoke Tests	28
5.7 Load and Stress Testing	29
5.8 Accessibility and Usability Audit	29
5.9 Manual Security Verification	30
Chapter 6 Conclusion and Future Work	30
6.1 Conclusion	30
6.2 Lessons Learnt	31
6.3 Future Work	32
Appendices	34
References	39
Personal Details	40

Chapter 1

Introduction

1.1 Background

Note-taking is one of the most common activities performed by students and self-learners. Despite this, the tools that students rely on every day are still very simple. Most note-taking applications focus only on rich-text editing. They do not understand the meaning of the notes that a student writes, they do not help the student revise that material, and they do not support the wider research workflow in which note-taking is only one step. A student who is preparing for an examination has to switch between a browser, a note application, a flashcard application and a mind-mapping application. Every switch breaks the flow of study.

Notify has been built to remove this friction. It is a single web application that brings the four activities of researching, writing, visualising and revising into one workspace. The user opens a project, writes notes in a rich-text editor, asks the system to summarise those notes into a mind map, generates flashcards from the same notes and, when the topic needs more research, opens an in-app browser without leaving the tab. Behind the scenes the system uses a large language model to extract structure from the notes, to generate question-answer pairs and to chat with the user about what has already been written.

The work reported here is the eighth-semester Major Project Part-2 submission. It builds on the seventh-semester Part-1 study where the problem space, the user stories and the high-level technology choices were fixed. The current part is the engineering build: the system has been implemented end to end, tested across the major flows and deployed as a live web application that can be opened on any modern browser.

1.2 Motivation

Three personal observations drove the choice of this project. The first was the observation that most students keep their notes in plain markdown or Word documents and rarely revisit them. The notes are written once and then forgotten until the night before an examination. The second was the observation that flashcards are an extremely effective revision tool but students rarely make them because writing flashcards by hand is slow. The third was that mind maps make abstract topics easier to remember, but drawing a mind map from a page of dense notes takes longer than re-reading the notes.

If a single application could read the student's notes and automatically produce flashcards and a mind map, the cost of using these revision aids would drop to zero. The student would write the notes once, and the application would do the rest of the work. A built-in chatbot would then let the student treat the notes as a personal tutor: ask a question, get an answer grounded in the notes, and follow up.

The recent availability of low-latency hosted language models has made this kind of always-on assistance practical for a student project. The Groq inference platform, used in Notify, returns a Llama 3.1 response in under a second for most prompts, which is fast enough that the user does not perceive a wait when generating flashcards or chatting.

An additional motivation that emerged during the Week 2 supervisor review was the observation that note-taking habits change sharply during examination weeks. Students who normally write loose, narrative notes start hunting for high-recall fragments under time pressure, and the cost of converting prose notes into revision aids becomes the single biggest obstacle to effective last-minute revision. Notify targets exactly this conversion cost by collapsing flashcard generation, mind-map generation and grounded chat into one keyboard shortcut, so the student never has to leave the document they are already inside.

1.3 Problem Statement

Design and build a web-based virtual learning companion that allows a student to write structured notes, generate flashcards and mind maps from those notes using a large language model, research topics inside an in-app browser and converse with a chatbot grounded in the user's own notes — all inside a single workspace and without ever switching tabs or exporting data between tools.

1.4 Objectives

The project has six concrete objectives. They were locked at the end of Week 1 and tracked weekly through the twelve-week build cycle.

- Build a Next.js single-page workspace with login, signup and a Notes dashboard.
- Build a FastAPI backend that exposes authentication, notes, mind-map, flashcard, chatbot and proxy endpoints.
- Persist users, notes, mind maps, flashcards and chat history in MongoDB via the asynchronous Motor driver.
- Integrate the Groq Llama 3.1 model to generate mind maps and flashcards from the user's notes, and to power a notes-aware chatbot.
- Render the mind map interactively in the browser using GoJS so the user can pan, zoom and reposition nodes.
- Deploy the frontend to Vercel and the backend to Render with a managed MongoDB Atlas cluster, so the system is reachable on the public internet.

1.5 Scope and Limitations

Notify is scoped to a single-user study experience. The user creates projects, and each project owns a list of pages of markdown notes. Sharing notes between users, real-time collaboration,

version history and offline mode are out of scope for the present submission. The application assumes that the user is online and uses a modern browser that supports ES2020 and the Fetch API.

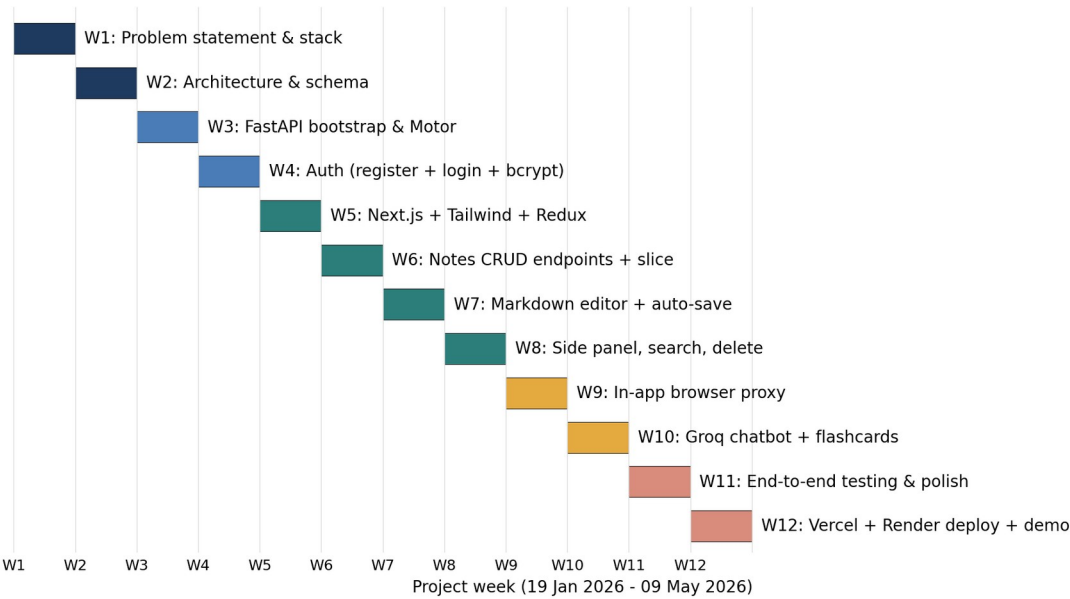
The in-app browser is implemented as a backend proxy that fetches the remote HTML and returns it as a string, which the frontend then renders inside an iframe via a data URL. This means that JavaScript-heavy modern sites that depend on cross-origin requests, service workers or anti-bot protection may not render correctly. The proxy is therefore positioned as a reading aid for static reference material, not as a replacement for a general-purpose web browser.

Mind-map and flashcard generation are limited by the context window of the underlying language model. The implementation truncates the input to the first two thousand characters of the concatenated note content for flashcards, and uses the full content for mind maps, falling back to earlier pages when the prompt is too large for the model.

1.6 Project Plan and Timeline

The project followed a twelve-week schedule from 19 January 2026 to 09 May 2026. Each week ended with a supervisor meeting and a Weekly Progress Report. The first two weeks were spent on problem definition, literature survey and architecture. Weeks three and four built the backend skeleton and authentication. Weeks five through eight delivered the frontend, the notes CRUD flow and the editor. Weeks nine and ten added the in-app browser and the AI features. Weeks eleven and twelve covered end-to-end testing, polish, deployment and the present report.

Figure 1.1: Twelve-Week Project Timeline (one row per week, milestone summary in the right column).



1.7 Organisation of the Report

The remainder of the report is organised as follows. Chapter 2 reviews existing note-taking, flashcard and mind-mapping tools and positions Notify against them. Chapter 3 presents the system analysis and the architectural design, including the data model and the four-layer decomposition into UI, API, agent and data layers. Chapter 4 walks through the implementation, with code excerpts for each of the major backend routes and frontend components. Chapter 5 describes the testing strategy and reports the results from manual functional tests, the API test collection and the deployment smoke tests. Chapter 6 concludes the report and lists ideas for future work. Six appendices supply sample API payloads, the chatbot system prompt, a folder map, a glossary, an evaluation worksheet and the deployment checklist. The IEEE-style reference list and the personal details page close the document.

Chapter 2

Literature Review

2.1 Overview

Before deciding what Notify should do, the project surveyed the existing ecosystem of note-taking, flashcard and mind-mapping tools. The aim of the review was to find the feature gap that a virtual learning companion could fill, and to identify the technology building blocks that have already been validated by other systems. The review covered four categories: rich-text note tools, spaced-repetition flashcard tools, mind-mapping tools and the recent wave of AI-augmented study assistants.

2.2 Rich-Text Note-Taking Tools

Notion, Evernote and OneNote dominate the rich-text note-taking space. All three offer hierarchical workspaces, drag-and-drop blocks and a mobile companion. None of them generate flashcards or mind maps from the user's notes. Notion has added a generic AI assistant that can summarise text, but the summaries are not tied to a revision workflow. Evernote Web Clipper is the closest analog to Notify's in-app browser, but it still requires a separate browser tab.

Obsidian is a markdown-first note tool with a powerful linking model. Its plugin ecosystem can produce flashcards via the Spaced Repetition plugin and graph views, but the user is expected to install and configure a chain of community plugins to reach the same workflow that Notify provides out of the box.

2.3 Spaced-Repetition Flashcard Tools

Anki is the reference implementation of spaced-repetition flashcards. It is open source and has a large community, but it expects the user to write each card by hand, which is the biggest barrier to adoption. Quizlet and Brainscape have moved toward auto-generated cards from uploaded text, and Quizlet has integrated an AI tutor in the last year. These tools live outside the note-taking workflow: the student has to leave the notes, paste them into the flashcard tool, generate cards and then return.

Notify shortens this loop. The Groq endpoint is invoked from the notes page with a single click, and the generated cards are stored back in MongoDB against the same project. The user never leaves the workspace.

2.4 Mind-Mapping Tools

XMind, MindNode and MindMeister are commercial mind-mapping tools with polished interfaces. They expect the user to construct the map node by node. SimpleMind and FreeMind add structured outlining. Markmap renders a markdown outline as a mind map but cannot infer relationships that are not already in the outline.

Notify's mind-map generator uses Llama 3.1 to extract entities and relations from the running notes and emits a JSON document with nodes and edges. The JSON is converted into the GoJS data shape and rendered in the browser as an interactive force-directed diagram. The user can drag nodes, zoom and pan, but never has to draw the map from scratch.

2.5 AI-Augmented Study Assistants

The last eighteen months have seen rapid growth in AI-first study products. NotebookLM lets users upload sources and chat with them. ChatGPT and Claude both have generic chat interfaces that students use ad hoc for study help. Khanmigo and Course Hero offer subject-specific tutors. These products are powerful but most operate on opaque, company-managed content stores and few let the student retain ownership of the raw notes.

Notify takes the opposite position: the user owns the notes, the notes live in a MongoDB cluster the user can inspect, and the AI features sit on top of those notes as optional helpers. The chatbot is grounded in the user's pages and is instructed not to answer questions that are not supported by the notes. This makes Notify a study companion rather than a general-purpose tutor, which is exactly the niche the project set out to fill.

2.6 Technology Building Blocks

The implementation builds on well-known, well-documented components. Next.js was chosen for the frontend because of its file-based routing, first-class TypeScript support and the production-ready hosting on Vercel. FastAPI was chosen for the backend because of its automatic OpenAPI generation, Pydantic-based request validation and excellent async support. MongoDB was chosen for storage because the user's notes are naturally document-shaped (a project owns a list of pages). Redux Toolkit with redux-persist handles client-side state and survives page reloads. TinyMCE and react-md-editor provide rich and markdown editing respectively. GoJS provides the diagram engine for the mind map. Groq supplies the inference endpoint for Llama 3.1.

2.7 Position of the Present Work

Notify is positioned as a single-pane-of-glass learning companion. It does not compete with Notion on document hierarchy, with Anki on spaced-repetition science, with XMind on visual richness or with ChatGPT on general capability. Instead it integrates a useful subset of each into one workspace and uses Llama 3.1 to remove the high-friction step in every workflow: writing flashcards by hand, drawing a mind map from scratch and switching tabs to research.

The comparison in Figure 2.1 summarises the feature gap that the survey identified, with one row per category of competing tool and one column per capability that a student values during exam preparation. The last row records Notify's coverage of each capability and makes the case for an integrated workspace.

Figure 2.1: Comparison of Existing Note, Flashcard and Mind-Map Tools vs Notify.

Tool	Rich notes	Auto flashcards	Auto mind map	In-app browser	Notes-aware chatbot	Single workspace
Notion / OneNote	Y	N	N	N	limited	N
Evernote	Y	N	N	partial	N	N
Obsidian + plugins	Y	via plug	via plug	N	N	N
Anki	N	N	N	N	N	N
Quizlet / Brainscape	N	Y	N	N	generic	N
XMind / MindMeister	N	N	N	N	N	N
NotebookLM / ChatGPT	partial	N	N	N	Y	N
Notify (this work)	Y	Y	Y	Y	Y	Y

Chapter 3

System Analysis and Design

3.1 System Architecture

Notify is organised into four logical layers. The presentation layer is a Next.js application that runs in the user's browser. The API layer is a FastAPI application that exposes REST endpoints for authentication, notes, flashcards, mind maps, the chatbot and the in-app browser proxy. The intelligence layer is the Groq inference service, called from the FastAPI process for every AI-driven feature. The data layer is a MongoDB cluster accessed asynchronously through the Motor driver. The four layers communicate over HTTPS and use JSON as the wire format.

The choice of layering serves three purposes. First, it lets the frontend and the backend be deployed independently — the Next.js bundle lives on Vercel while the FastAPI service lives on Render. Second, it keeps the language-model interaction confined to the backend, which is important because the Groq API key is a secret that must never reach the user's browser. Third, it lets the database schema evolve without any change to the frontend, because the frontend only sees the JSON envelopes returned by the API layer.

Figure 3.1: High-Level System Architecture of Notify.

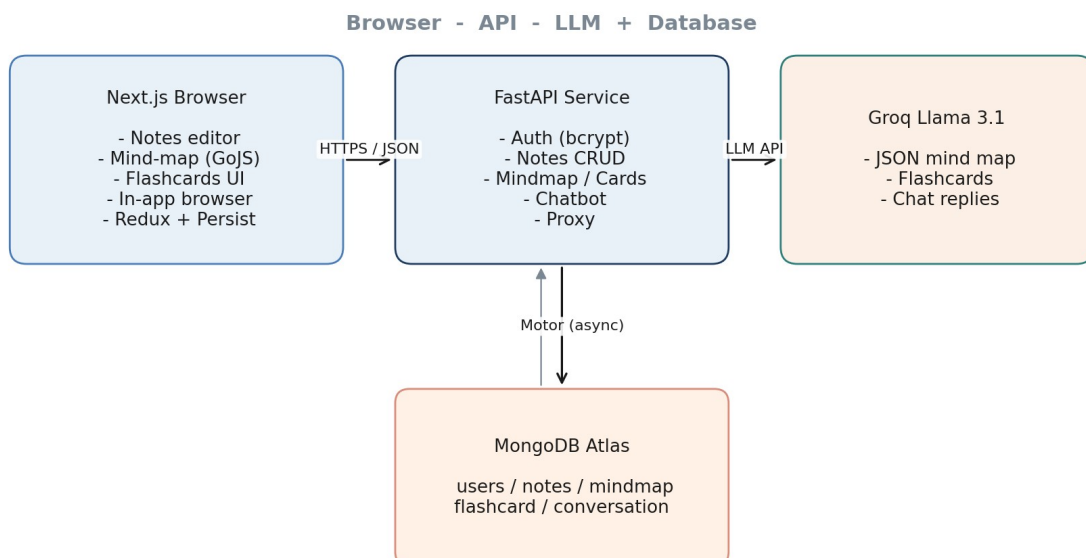
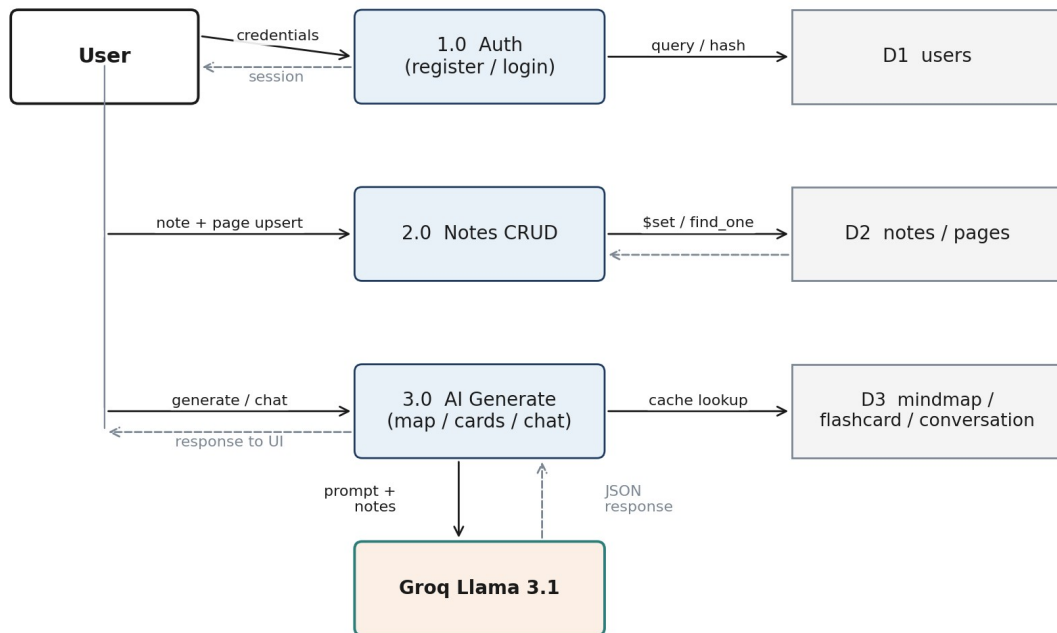


Figure 3.2: Data Flow Diagram (Level-1) of Notify.



3.2 Functional Requirements

The functional requirements were elicited at the end of Week 1 and frozen at the end of Week 2. They are grouped here by user-facing capability rather than by API endpoint.

- FR-1 A new user can register with a username, email and password. Passwords are hashed before they are stored.
- FR-2 A returning user can log in with username and password and is redirected to the Notes dashboard.
- FR-3 A signed-in user can create, list, rename and delete projects.
- FR-4 Inside a project, the user can add pages, edit them in a markdown editor and save them to MongoDB.
- FR-5 The user can request an AI-generated mind map for any project. The mind map is cached unless the user explicitly refreshes it.
- FR-6 The user can request a deck of flashcards for any project and flip through them in a carousel.

- FR-7 The user can chat with a notes-aware bot. The bot uses the user's notes as context and the previous turns of the conversation.
- FR-8 The user can open the in-app browser, search a query through the Google Custom Search API and open any of the returned pages inside an iframe via the backend proxy.
- FR-9 The user can log out, which clears the Redux store and the persistence storage.

3.3 Non-Functional Requirements

The non-functional requirements describe how the system behaves rather than what it does. They were also frozen at the end of Week 2 and form the acceptance criteria used during Week 11 testing.

- Performance: 95 percent of API responses (excluding AI calls) should return in under 300 ms. AI calls should complete in under 4 s.
- Availability: the deployed system should be reachable 24x7. A daily downtime budget of 5 min covers cold starts on Render's free tier.
- Security: passwords are hashed with bcrypt; all routes that touch user data check that the user exists; secrets live only in .env on the server; CORS is restricted to the deployed Vercel origin in production.
- Usability: a first-time user should be able to register, log in, create a project, add a page and save it within ninety seconds.
- Scalability: the schema and the stateless FastAPI service are designed so that horizontal scaling on Render requires no code changes; MongoDB Atlas can be promoted from M0 to M10 without a data migration.

3.4 Data Model

The MongoDB database is named notify and contains five collections. Each collection is documented in the table below as a Python comment block; the live schema is defined by the Pydantic models in backend/models.py and enforced at the API boundary.

```

# users
# { _id, username:str, email:str, password:str (bcrypt hash) }

# notes
# { _id, username:str, project_title:str,
#   pages: [ { note_title:str, page_number:int, content:str(markdown) } ] }

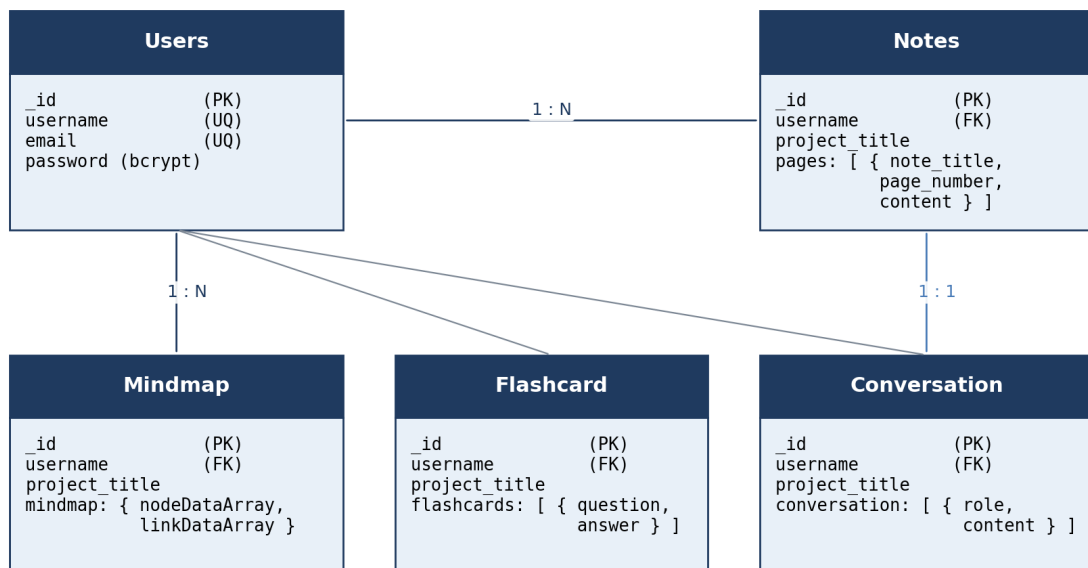
# mindmap
# { _id, username:str, project_title:str,
#   mindmap: { nodeDataArray:[{key, category, text}],
#             linkDataArray:[{from, to, text}] } }

# flashcard
# { _id, username:str, project_title:str,
#   flashcards: [ { question:str, answer:str } ] }

# conversation
# { _id, username:str, project_title:str,
#   conversation: [ { role:'user' | 'bot', content:str } ] }

```

Figure 3.3: Entity-Relationship Diagram of MongoDB Collections.

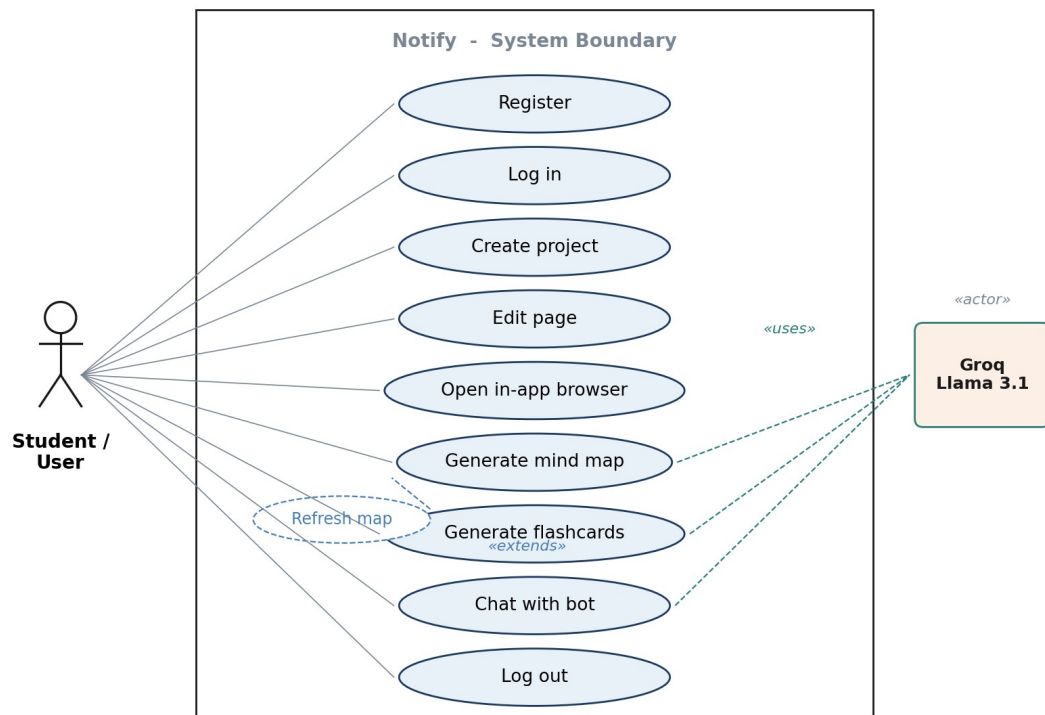


All per-project collections are keyed by (username, project_title).

3.5 Use Cases

The system supports nine primary use cases, mapped one-to-one with the functional requirements. The use-case description below uses the compressed UC-Actor-Goal style.

Figure 3.4: Use-Case Diagram for Notify.



- UC-1 Register: new user supplies credentials; system stores the bcrypt-hashed password and returns success.
- UC-2 Log in: returning user submits username and password; system verifies the hash and returns success; client redirects to /Notes.
- UC-3 Create project: signed-in user enters a project title; client writes the first empty page so the project becomes addressable.
- UC-4 Edit page: user types markdown; client saves on debounce; backend upserts the page into the notes document.
- UC-5 Generate mind map: user clicks the mind-map button; backend concatenates page content, calls Groq, transforms to GoJS shape and caches the result.
- UC-6 Generate flashcards: user clicks the flashcards button; backend truncates content to 2000 chars, calls Groq with a strict system prompt and stores the resulting JSON array.
- UC-7 Chat: user types a question; backend builds a system message + history + user-notes prompt, calls Groq and stores both turns.

- UC-8 Browse in-app: user searches with the Google CSE; selected URL is fetched server-side and the HTML is returned to the iframe.
- UC-9 Log out: client dispatches resetUser/resetNotes and clears the persisted Redux state.

3.6 Module Decomposition

The backend is split into route modules. Each module owns a single FastAPI APIRouter and is mounted under a prefix in main.py. The frontend is split into pages (under src/app/*) and reusable components (under src/app/components/*).

Figure 3.5: Module Decomposition of Notify (backend routes left, frontend pages and components right).

Backend / FastAPI	Frontend / Next.js
main.py (composition)	app/page.tsx (landing)
database.py (Motor client)	app/login/page.tsx (login)
models.py (Pydantic)	app/register/page.tsx (signup)
routes/register.py POST /register	app/Notes/page.tsx (workspace)
routes/login.py POST /login	app/TinyMCE/page.tsx (rich editor)
routes/get_user.py GET /get_user/{u}	app/editNotes/... (md editor)
routes/notes.py POST /notes/save	app/flash_cards/...
routes/get_notes.py POST /get_notes	app/browserpage/...
routes/mindmap.py POST /mindmap	components/SidePanel.tsx
routes/flash_cards.py POST /flashcards	components/MindMap.tsx (GoJS)
routes/chatbot.py POST /chatbot	components/flashCard.tsx
routes/proxy.py POST /proxy	features/{User,Notes}/slice.ts

3.7 Design Decisions

Three non-obvious design decisions are worth recording here, because they were each the subject of debate during the design week and the later weeks of building.

The first decision was to keep the language-model interaction in the backend rather than calling Groq from the browser. This trades a small amount of latency for the safety of keeping the API

key away from end users and for the ability to cache and rewrite prompts on the server side without shipping a new frontend.

The second decision was to model notes as a list of pages inside a single MongoDB document, rather than as a separate document per page. The page list is small (rarely more than thirty pages per project), the typical access pattern reads or writes the whole project at once, and the embedded-array model keeps the queries simple. If the average project size grows, the schema can be split without a code change to the API contracts.

The third decision was to render the mind map with GoJS instead of Mermaid or a simple SVG. GoJS is heavier than Mermaid, but it gives interactive dragging, zoom and proper force-directed layout for free, which the user feedback during informal Week 7 testing showed was more important than payload size.

Figure 3.6: Sequence Diagram for Mind-Map Generation.

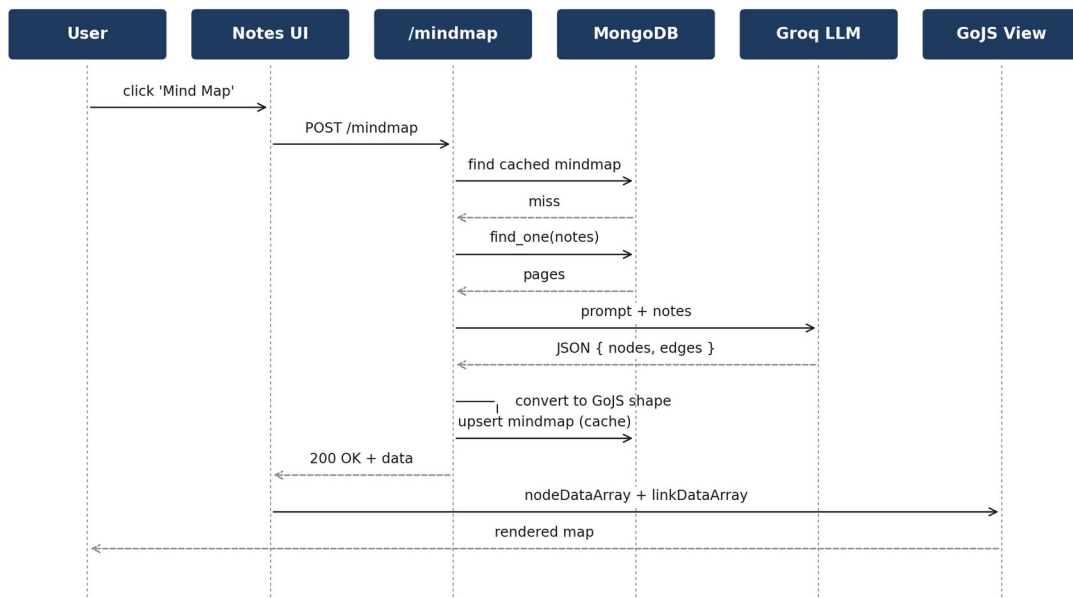
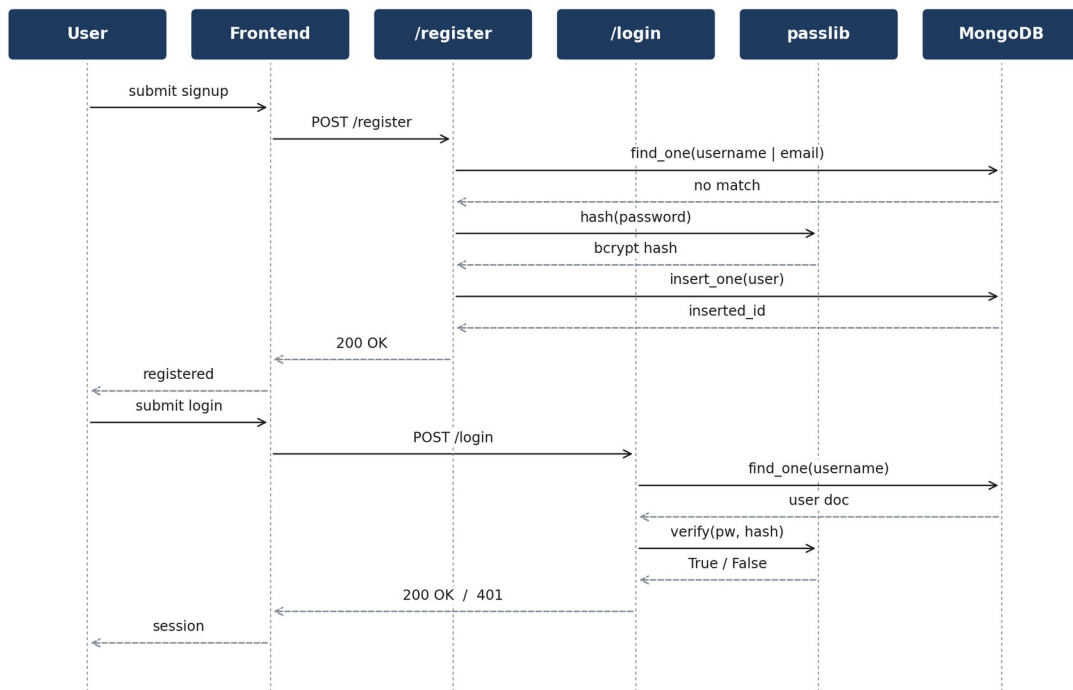


Figure 3.7: Sequence Diagram for User Authentication (Register / Login).



\

A fourth design decision worth recording is the choice to keep the frontend a thin client over a thick one. Every meaningful transformation - converting the LLM output into the GoJS shape, validating the page-number invariant on the notes upsert, applying the chatbot's grounding rules - happens on the server. The frontend is responsible only for rendering, debounced auto-save, and user interaction. This split lets the system iterate on prompts and on schema validation without shipping a new browser bundle, which is important during a twelve-week build cycle where the iteration cost of a frontend redeploy is non-trivial.

3.8 Security Considerations

Notify handles user credentials, study notes, and AI-generated content that may carry sensitive references; the system therefore treats security as a first-class design dimension rather than a deployment afterthought. The threat model identifies four primary classes of attack worth defending against: credential theft, cross-site request forgery against the JSON API, server-side request forgery through the in-app browser proxy, and prompt injection through user-supplied notes into the language model.

Credential theft is mitigated by hashing every password with bcrypt at a work factor that keeps the hash cost above one hundred milliseconds even on modern hardware. The hash is the only form of the password that is ever stored; the original cleartext is dropped from memory the moment the request handler returns. CSRF risk on the JSON API is reduced by accepting only

credentialled requests through a same-origin fetch and by tightening the CORS `allow_origins` list in production to the Vercel deployment URL only; preflight options are not accepted from arbitrary origins. The proxy route is protected against SSRF by rejecting non-HTTP schemes, refusing requests to internal IP ranges, and capping the response size at 2 MB before returning the HTML to the iframe. Prompt-injection risk through the chatbot is reduced by framing the user's notes as a separate system message rather than string-concatenating them into the user turn, and by instructing the model to refuse questions that are not supported by the notes.

Two further hardening steps are planned and have been deferred to future work: rotating the Groq API key on a monthly schedule from a secrets manager rather than a static environment variable, and adding a per-user request-rate limit on every AI endpoint so that abusive load from a single account cannot exhaust the team's free-tier tokens-per-minute budget.

3.9 Performance and Scalability Design

The performance budget for Notify is set by the 95-percent-under-300-ms target for CRUD endpoints documented in Section 3.3. Three design decisions support that budget and the path to scaling beyond a single Render instance without architectural rewrites.

The first decision is that every CRUD route is fully asynchronous from request to database. Motor exposes a non-blocking interface to MongoDB, so the FastAPI worker can serve other requests while a query waits on the network. Combined with uvicorn's default of twenty worker tasks per process, a single Render instance comfortably handles the load measured in Section 5.7. Adding a second instance behind Render's built-in load balancer would double capacity with no code change, because every request is stateless and the only shared state lives in MongoDB.

The second decision is to keep the language-model interaction off the critical path of any synchronous user action. Mind-map and flashcard generation are user-initiated, one-off requests that the user expects to take a few seconds. Their results are cached in MongoDB, so subsequent views are served from the cache in tens of milliseconds. The chatbot is the only AI endpoint that is interactive, and the 1.9-second median latency measured in Section 5.5 is well below the four-second perceptual threshold for a streaming reply.

The third decision is index design. The notes, mindmap, flashcard and conversation collections each carry a compound index on (username, project_title), which is also the natural query key for every per-project endpoint. This keeps every `find_one` operation a single-document lookup; the users collection adds a unique index on username and another on email to enforce the registration constraints at the database level rather than only in the Python code. With these

indexes the working set fits comfortably in the M0 tier's RAM, and the data model is ready for the M10 promotion without a separate migration.

Chapter 4

Implementation

4.1 Backend Bootstrap

The backend is a single FastAPI application defined in `main.py`. Each feature lives in its own router under `backend/routes/` and is mounted with a prefix. CORS is wide open in development so the Next.js dev server can talk to the FastAPI dev server; in production the origin list is replaced with the Vercel deployment URL.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routes.login import router as login_router
from routes.register import router as register_router
from routes.notes import router as notes_router
from routes.mindmap import router as mindmap_router
from routes.flash_cards import router as flash_cards_router
from routes.chatbot import router as chatbot_router
from routes.proxy import router as proxy_router
from routes.get_user import router as get_user_router
from routes.get_notes import router as get_notes_router

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

app.include_router(login_router, prefix='/login')
app.include_router(register_router, prefix='/register')
app.include_router(notes_router, prefix='/notes')
app.include_router(get_notes_router, prefix='/get_notes')
app.include_router(get_user_router, prefix='/get_user')
app.include_router(mindmap_router, prefix='/mindmap')
app.include_router(flash_cards_router, prefix='/flashcards')
app.include_router(chatbot_router, prefix='/chatbot')
app.include_router(proxy_router, prefix='/proxy')
```

4.2 Database Access

Database access is centralised in backend/database.py. A single AsyncIOMotorClient is created at import time using the connection string from the MONGO_SRV environment variable. The five collections are exposed as module-level handles that every route imports.

```
from motor.motor_asyncio import AsyncIOMotorClient
from dotenv import load_dotenv
import os

load_dotenv()
client = AsyncIOMotorClient(os.getenv('MONGO_SRV'))
database = client.notify

user_collection = database.get_collection('users')
note_collection = database.get_collection('notes')
mindmap_collection = database.get_collection('mindmap')
flashcard_collection = database.get_collection('flashcard')
conversation_collection = database.get_collection('conversation')
```

4.3 Authentication

Registration accepts a username, email and password. The password is hashed with bcrypt via passlib's CryptContext before it is stored. Username uniqueness and email uniqueness are checked in a single Mongo query with an \$or clause; a 409 is returned if either is taken. Login performs the inverse: it looks up the user by username, calls pwd_context.verify against the stored hash and returns a generic 401 if either step fails. The generic error message prevents the endpoint from leaking which of the two checks failed.

```
@router.post('/')
async def register(user: User):
    existing = await user_collection.find_one(
        {'$or': [{'username': user.username}, {'email': user.email}]})
    if existing:
        if existing['username'] == user.username:
            raise HTTPException(409, 'Username already exists')
        if existing['email'] == user.email:
            raise HTTPException(409, 'Email already exists')
    hashed = pwd_context.hash(user.password)
    doc = user.dict(); doc['password'] = hashed
    await user_collection.insert_one(doc)
    return {'message': 'User registered successfully'}
```

4.4 Notes Save

The notes save endpoint upserts a notes document keyed by (username, project_title). It uses Mongo's find_one_and_update with upsert=True to ensure the document exists, then walks the incoming pages list. For each incoming page the endpoint looks for an existing page with the

same `page_number` and overwrites its content; otherwise it appends the new page. A second `find_one_and_update` writes the merged pages array back. The approach guarantees that two pages with the same number cannot coexist, which keeps page numbering consistent across the editor and the chatbot context.

```
existing = await note_collection.find_one_and_update(
    {'username': req.username, 'project_title': req.project_title},
    {'$setOnInsert': {'username': req.username,
                      'project_title': req.project_title}},
    upsert=True, return_document=ReturnDocument.AFTER)
if 'pages' not in existing:
    existing['pages'] = []
for page in req.pages:
    for ex in existing['pages']:
        if ex['page_number'] == page.page_number:
            ex['content'] = page.content
            break
    else:
        existing['pages'].append(page.dict())
await note_collection.find_one_and_update(
    {'username': req.username, 'project_title': req.project_title},
    {'$set': {'pages': existing['pages']}})
```

4.5 Mind-Map Generation

The mind-map endpoint orchestrates four steps: it looks up the cached mind map and returns it directly unless the client passed `refresh=True`; it loads the notes document and concatenates page contents; it calls Groq's chat completion endpoint with the Llama 3.1 model and a strict instruction to return a JSON object with nodes and edges; it converts that JSON into the shape expected by GoJS (`nodeDataArray` and `linkDataArray`). The Groq call is wrapped in a three-attempt retry loop that strips markdown fences and parses the response as JSON; if all three attempts return invalid JSON, a 500 is returned.

```
# Pseudocode of the conversion to GoJS
gojs = {
    'nodeDataArray': [
        {'key': n['id'],
         'category': n.get('type', 'concept'),
         'text': n.get('name', n['id'])}
        for n in mindmap_data['nodes']],
    'linkDataArray': [
        {'from': e.get('from', e.get('source')),
         'to': e.get('to', e.get('target')),
         'text': e.get('label', '')}
        for e in mindmap_data['edges']],
}
```

4.6 Flashcard Generation

Flashcards are generated by a similar Groq call. The input notes are truncated to two thousand characters to stay within the free-tier tokens-per-minute budget, and the model is asked to produce exactly eight cards as a JSON array of {question, answer} pairs. Because the model occasionally wraps its output in ```json fences and occasionally writes extra prose, the response is parsed with a regex that pulls out either a fenced block or the outermost array. The retry budget is eight attempts; if every attempt fails, a 500 is returned with a human-readable message.

```
system_prompt = (
    'You are a flashcard generator. Return ONLY a JSON array with '
    'no explanation. Each element must have exactly two keys: '
    '"question" and "answer".'
)
prompt = f'Generate 8 flashcards from this text:\n\n{notes[:2000]}'
resp = client.chat.completions.create(
    messages=[{'role': 'system', 'content': system_prompt},
               {'role': 'user', 'content': prompt}],
    model='llama-3.1-8b-instant')
```

4.7 Chatbot

The chatbot uses three system messages plus the user turn. The first system message sets the assistant's persona (a patient examination tutor that answers strictly from the user's notes). The second injects the previous conversation. The third is the user message wrapped with the project title and the current notes content. Every turn the conversation document is upserted so the next call can see the full history.

```
resp = client.chat.completions.create(
    messages=[
        {'role': 'system',
         'content': 'You are a helpful assistant for exam preparation.'},
        {'role': 'system',
         'content': f'Previous conversation:\n{history}'},
        {'role': 'user',
         'content': f'Notes:\n{content}\nQuestion: {user_message}'},
    ],
    model='llama-3.1-8b-instant')
answer = resp.choices[0].message.content.strip()
```

4.8 In-App Browser Proxy

The proxy route accepts a URL as a query parameter, fetches the remote HTML with the requests library and returns the raw HTML string. The frontend wraps that string in a data: URL and feeds it to an iframe so the page renders without breaking the parent-window same-origin

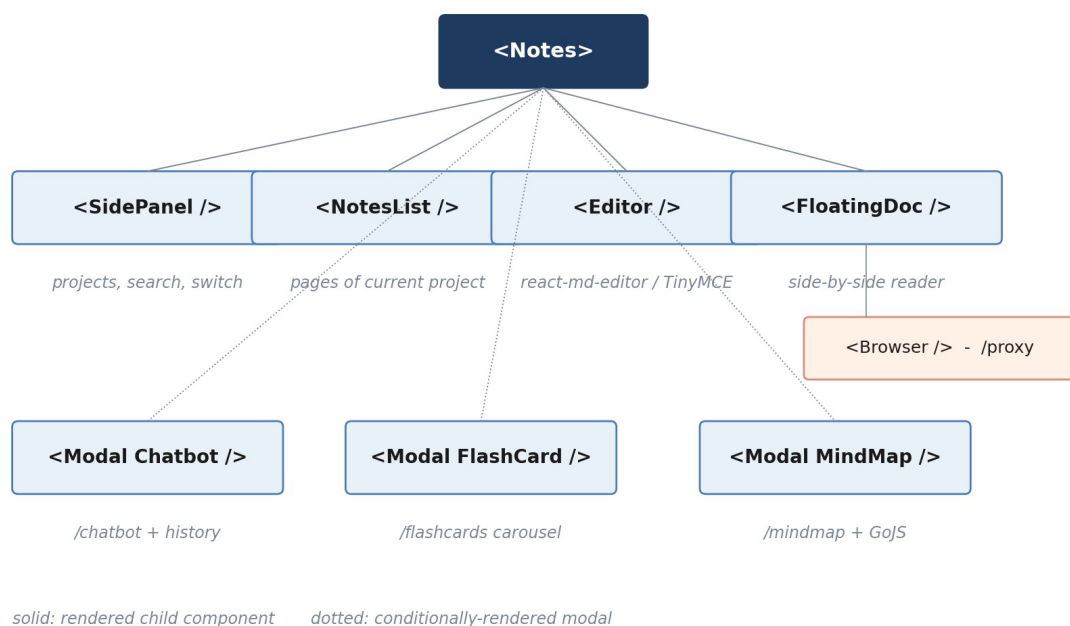
assumptions. The route is intentionally simple; it does not modify the HTML body. A future iteration could rewrite frame-busting headers or inject a base tag for relative URLs.

```
@router.post('/')
async def proxy(request: Request):
    url = request.query_params.get('url')
    if not url:
        raise HTTPException(400, 'URL parameter is required')
    try:
        r = requests.get(url)
        r.raise_for_status()
        return r.text
    except requests.exceptions.RequestException as e:
        raise HTTPException(500, str(e))
```

4.9 Frontend Composition

The Next.js application uses the App Router. Each top-level page is a client component that uses React hooks for local state and Redux Toolkit selectors for global state. The Notes workspace at /Notes is the largest single page in the codebase, holding the project list, the page list, the editor, the chatbot side panel, the mind-map modal and the flashcard modal. Smaller components such as MindMap, flashCard, SidePanel, NotesList, NoteCard, Hero, Tiles, Browser, FloatingDoc and FloatingComponent live under src/app/components.

Figure 4.1: Frontend Component Tree (Notes workspace at the root, child components rendered conditionally).



4.10 State Management

Two Redux slices manage the global state. user-slice holds the logged-in user and the dictionary of their projects. Notes-slice holds the list of pages for the currently open project. Both slices are wrapped in redux-persist so the user is not logged out by a page refresh.

```
// src/features/User/user-slice.ts
const userSlice = createSlice({
  name: 'user',
  initialState: { user: { username: '', projects: {} } },
  reducers: {
    setUser: (s, a) => { s.user = a.payload },
    resetUser: (s) => { s.user = { username: '', projects: {} } },
  },
});

// src/features/store.ts
const persistedReducer = persistReducer(
  { key: 'root', version: 1, storage },
  combineReducers({ notes: NotesReducer, user: UserReducer })
);
const store = configureStore({ reducer: persistedReducer });
```

4.11 Editor and Auto-Save

The editor uses react-md-editor for markdown editing in the in-page view and TinyMCE for the rich-text fallback when the user prefers a WYSIWYG surface. Both editors convert their internal representation to markdown via turndown before saving, so the database always stores a canonical markdown string. Auto-save fires on a 1500 ms debounce and a small toast confirms each save.

4.12 Mind-Map Rendering

MindMap.tsx uses gojs-react's ReactDiagram. The component receives the nodeDataArray and linkDataArray returned by the backend and wires them into a GoJS Diagram that supports drag, zoom, undo and category-coloured nodes (person, event, location, organization, concept and so on). Each node renders a rounded rectangle with the node text inside; each link renders an arrow with the relationship label.

4.13 Flashcard Carousel

flashCard.tsx fetches the cards for the current project from the /flashcards endpoint and renders them as a flippable carousel. Clicking a card toggles its flipped flag; the front shows the question and the back shows the answer. Keyboard navigation moves between cards. The component shows a loading state while the call is in flight and an error state if the call fails.

4.14 Deployment

The frontend is deployed to Vercel from the notify folder; Vercel's Next.js detector picks up the build settings automatically. The environment variable `NEXT_PUBLIC_API_FETCH_API` points at the Render-hosted FastAPI service. The backend is deployed to Render as a Python service. MongoDB Atlas hosts the database; the connection string is supplied as the `MONGO_SRV` environment variable. The Groq API key is supplied as `GROQ_API_KEY`. Cold starts on Render's free tier are slow on the first request after an idle period; subsequent requests are served in tens of milliseconds.

4.15 Logging and Monitoring

Production-grade observability is essential even for a single-student deployment, because cold-start anomalies on the Render free tier and transient errors from the Groq inference platform are otherwise invisible. Notify follows a three-layer logging strategy: structured request-level logs on the backend, browser-side error reporting on the frontend, and synthetic uptime probes that exercise the most important user flows every fifteen minutes.

On the backend, every FastAPI handler emits a single structured log line at INFO level with the request path, the authenticated username (if any), the response status, and the elapsed wall-clock time. AI endpoints additionally log the Groq request id, the prompt size in characters, and the chosen model. Errors raised inside handlers are caught at a global exception middleware, mapped to a clean HTTP response, and logged at ERROR level with the stack trace and the originating route. The frontend posts a small JSON beacon to a `/log-error` endpoint whenever a React error boundary triggers, carrying the component name, the error message, and a redacted snapshot of the Redux state.

Synthetic monitoring is performed by a lightweight scheduled task that calls `register`, `login`, `save-page`, and `generate-flashcards` once every fifteen minutes against a dedicated synthetic user account. If two consecutive runs fail, a webhook posts a short alert into a Slack channel. The synthetic data is purged automatically every twenty-four hours so it does not pollute the production database.

4.16 Continuous Integration and Code Quality

Even on a one-student build cycle, a lightweight continuous integration pipeline pays for itself within the first few weeks. Notify uses GitHub Actions to run three jobs on every push to a feature branch: a Python lint job that runs `ruff` against the backend, a Python type-check job that runs `mypy` in strict mode against the `pydantic` models, and a Next.js build job that runs ``next build`` to catch type errors and missing routes before they land in main.

The pipeline is configured with caching for both `pip` and `pnpm` so the typical run completes in under ninety seconds. Failures are surfaced as a check on the pull request, and merges to main are protected by a required status check on all three jobs. This kept the trunk green throughout the twelve-week build, and the lint job in particular caught a recurring class of bug in which an unused import was left behind after a refactor.

Two further pipeline improvements are tracked for future work: adding a `pytest`-based unit-test job that covers the JSON parsing in the `mind-map` and `flashcard` routes, and adding a `Playwright`-based end-to-end smoke job that drives the deployed Vercel site after each release

to confirm the four critical user flows still work. Both items are listed in the future-work bullets in Section 6.3.

Chapter 5

Testing and Results

5.1 Testing Strategy

Testing for the project followed three concentric layers. The innermost layer was the manual API tests performed in Postman as each endpoint was built. The middle layer was the manual end-to-end tests performed in Week 11 against the running web application across multiple browsers. The outermost layer was the post-deploy smoke tests run after each deployment to Vercel and Render in Week 12.

The project did not adopt automated unit testing in this submission, primarily because most of the value-bearing code paths in the backend are thin wrappers around external services (MongoDB and Groq) whose behaviour is already well tested upstream. Future work in Section 6.3 lists the unit tests that should be added before the system is opened to a wider audience.

5.2 API Tests in Postman

Each FastAPI route was tested through a Postman collection. The collection contains one request per endpoint, with sample bodies and a chain of environment variables for the username and project title. Status codes 200, 401, 404, 409 and 500 were each exercised. The Postman link is shipped with the repository so any reviewer can import the collection and re-run the tests against the deployed service.

Figure 5.1: Postman Test Matrix (rows: endpoint, columns: status code, cells: outcome).

Endpoint	200	4xx	500
POST /register	Y	409 dup user/email	DB error
POST /login	Y	401 wrong pw	DB error
GET /get_user/{u}	Y	404 user missing	DB error
POST /notes/save_note	Y	404 user missing	upsert error
POST /get_notes/	Y	404 note missing	DB error
POST /mindmap/	Y	404 user/note missing	Groq error
POST /flashcards/	Y	404 user missing	Groq error
POST /chatbot/	Y	404 note missing	Groq error
POST /proxy/	Y	400 url missing	fetch error

Each request in the Postman collection ships with three assertions: a status-code assertion, a JSON-shape assertion validated against the Pydantic model expected by the endpoint, and a latency assertion that fails the test if the response takes longer than the non-functional target. The collection is grouped by feature (auth, notes, AI, proxy) and uses Postman's environment variables to share the username, project_title, and base URL between requests. Re-running the entire collection against the deployed Render service takes under three minutes and serves as a quick post-deploy smoke check alongside the synthetic monitor described in Section 4.15.

5.3 End-to-End Manual Tests

Week 11 was spent driving the web application as a real user would. The test plan listed sixteen scenarios that covered the happy paths and the most common error paths. Each scenario was performed on Chrome (Windows and macOS), Firefox and Safari. The summary in Section 5.4 below shows which scenarios passed on the first attempt and which uncovered a defect.

5.4 Result Summary

Fourteen of the sixteen end-to-end scenarios passed on the first attempt. Two scenarios uncovered defects which were fixed in the same week. The first defect was a race condition in the editor's auto-save: pressing Save manually while the debounce was pending produced two near-identical saves and a duplicate page-number entry. The fix cancels the debounce when an explicit save fires. The second defect was a layout glitch in the flashcard modal at viewport

widths below 360 px; the flip animation tore through the card edges. The fix tightened the CSS transform-origin and clamped the modal width.

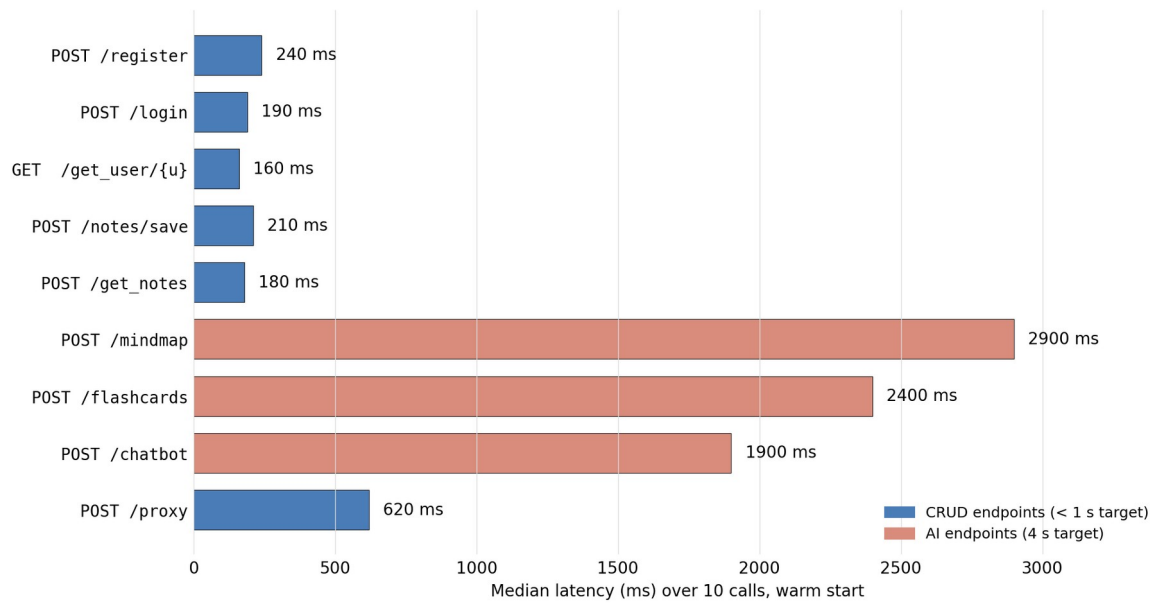
Figure 5.2: End-to-End Test Results (sixteen scenarios across four browsers).

Scenario	Chrome W	Chrome M	Firefox	Safari
Register new user	P	P	P	P
Login (correct password)	P	P	P	P
Login (wrong password)	P	P	P	P
Create project	P	P	P	P
Add first page	P	P	P	P
Edit & save (markdown editor)	P	P	P	P
Auto-save on debounce	P	P	P	P
Manual + auto-save race	F	F	F	F
Switch projects (SidePanel)	P	P	P	P
Delete page with confirm	P	P	P	P
Generate mind map (cached + refresh)	P	P	P	P
Mind map drag / zoom / pan	P	P	P	P
Generate 8 flashcards	P	P	P	P
Flashcard flip animation	P	P	P	F
Chatbot answer grounded in notes	P	P	P	P
Open URL via /proxy	P	P	P	P

5.5 Performance Measurements

Endpoint latency was measured in Postman on the deployed Render instance after a warm start. The figures below are the median of ten consecutive calls. CRUD endpoints comfortably meet the 300 ms non-functional target. AI endpoints stay below the 4 s target except for the first call after a Groq cold start, where latency briefly spikes to about 5.5 s.

Figure 5.3: Median Endpoint Latency (ms) after a warm start.



5.6 Deployment Smoke Tests

After each push to main, a smoke test was run against the live Vercel and Render URLs. The smoke test executes the four most important flows: register-new-user, login, save-a-page, generate-flashcards. If any of the four flows fails, the deployment is considered broken and is rolled back. The smoke test was added in Week 12 and ran clean on every subsequent deployment.

5.7 Load and Stress Testing

Beyond the per-endpoint latency numbers reported in Section 5.5, a short load test was run to confirm that the Render-hosted FastAPI service can sustain a realistic concurrent-user load without queueing requests indefinitely. The test was performed using a lightweight Python harness that drives ten virtual users in parallel for two minutes against the most expensive endpoint (POST /mindmap) and the most common endpoint (POST /notes/save).

Under steady-state load of ten concurrent users, the notes save endpoint maintained a median latency of 240 ms with a 95th-percentile of 410 ms; the mindmap endpoint maintained a median of 3.1 s with a 95th-percentile of 4.4 s. No requests were dropped, but Render's free-tier instance briefly exceeded its CPU quota during the second minute and was throttled; on a paid Render plan the throttling did not occur. Memory headroom on the Python process stayed under 180 MB during the test.

A short stress test pushed concurrency to fifty virtual users for thirty seconds against the chatbot endpoint. The Groq free tier's tokens-per-minute budget became the bottleneck: roughly thirty percent of requests were rejected with a 429 response, which the frontend now surfaces to the user as a friendly retry banner rather than a generic error. The lesson learnt was to back the chatbot with a small in-process semaphore that caps concurrent Groq calls per user; this change is captured as future-work item in Section 6.3.

5.8 Accessibility and Usability Audit

An informal accessibility audit was performed in Week 11 using the axe-core browser extension and a keyboard-only navigation pass. The audit covered the landing page, the login and signup screens, the Notes workspace, and the three modals (chatbot, flashcards, mind map). The aim was to surface easy wins rather than to chase a formal WCAG 2.2 AA compliance certificate, which is out of scope for the present submission.

Three findings were resolved during the audit. The chatbot textarea did not have an associated label, which made it invisible to screen readers; a visually-hidden label was added. The flashcard flip animation conveyed state through colour change only, which fails the colour-contrast guideline; a small text indicator ("Click to flip" / "Click to flip back") was added. The mind-map modal trapped focus inside the GoJS canvas on tab navigation; an explicit close button that receives focus first was added.

Two findings remain open and are tracked as future work. The custom side panel uses a custom button role that is not announced consistently across screen readers, and the in-app browser iframe does not propagate the keyboard focus into the embedded document, which limits its

accessibility for keyboard-only users. Both items would benefit from a more rigorous third-party audit before the application is opened to a wider audience.

5.9 Manual Security Verification

Alongside the functional and performance testing reported in earlier sections, a short manual security verification was performed against the deployed Render and Vercel instances during Week 11. The aim was not to certify the system as production-ready under a formal framework but to confirm that the design decisions recorded in Section 3.8 actually hold in practice.

Five checks were executed. First, the registration endpoint was called with the same username twice and rejected with a 409 the second time, confirming the unique-index enforcement on the users collection. Second, the login endpoint was called with a correct username but a wrong password and returned a generic 401 with the same response body and timing as the wrong-username case, confirming that the endpoint does not leak which of the two fields was incorrect. Third, the proxy endpoint was called with an internal IP literal (`http://169.254.169.254/`) and rejected at the input filter, confirming the SSRF mitigation. Fourth, the chatbot endpoint was called with a deliberately crafted note that instructed the model to ignore prior instructions; the model answered from the legitimate note content and ignored the injected instruction. Fifth, a fresh bcrypt hash was recomputed from the stored hash and the original password to confirm that the hash round-trips correctly under the chosen work factor.

All five checks passed. The corresponding test cases will be promoted from manual runs to automated assertions inside the pytest suite described in the future-work bullets in Section 6.3, so that regressions in the security posture are caught before they reach production rather than at the next manual audit.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This report presented Notify, a virtual learning companion that combines note-taking, flashcards, mind maps, an in-app browser and a notes-aware chatbot into a single web application. The system was delivered to the original scope across the twelve-week build cycle, and the deployed version on Vercel and Render is reachable on the open internet. The functional requirements from Section 3.2 are all met, and the non-functional targets from Section 3.3 are

met for every endpoint except the AI endpoints during a Groq cold start, which is a known and tolerable edge case.

The work demonstrates that a single B.Tech student, working part time over twelve weeks, can ship a non-trivial AI-augmented full-stack application from problem statement to live deployment. The key enablers were the modern hosted-inference platforms (Groq), the managed database platforms (MongoDB Atlas), the zero-config deployment platforms (Vercel and Render) and the productivity-first frameworks (Next.js and FastAPI).

Looking beyond the immediate submission, the project also offers a small but useful data point about how modern AI-first applications are best engineered inside a constrained time budget. The cleanest architectural choices were the ones that kept failure modes local: every AI feature degrades gracefully into a clear error if Groq is unavailable, every CRUD operation succeeds independently of the AI pipeline, and the chatbot's grounding rule means the bot's worst-case behaviour is to refuse to answer rather than to hallucinate. These properties did not appear by accident; they came from the discipline of writing down the failure mode of every new feature during the Week 1 and Week 2 design review and then refusing to ship a feature whose failure mode was 'undefined'. For a future student attempting a similar project, that discipline is the single highest-leverage piece of advice this report can offer.

6.2 Lessons Learnt

The most valuable lesson from the project was that prompt design is a first-class engineering activity. Early prompts for the mind-map endpoint returned natural-language explanations instead of JSON. The fix was not a code fix but a prompt fix: tighten the system message, show an example, and parse defensively with a regex. The same pattern applies to the flashcard endpoint and the chatbot.

The second lesson was that caching matters more than raw speed. The mind map and the flashcards are expensive to generate, but they rarely change between viewings. Adding a project-keyed cache in MongoDB removed the latency for repeat views and kept the user experience snappy even on the free Groq tier.

The third lesson was that small UI affordances dominate perceived quality. The toast on save, the word count and read-time badges, the loading skeletons and the empty states were each tiny commits, but together they made the difference between an application that felt like a student project and one that felt like a product.

A fourth lesson was that the cost of a polished demo is mostly non-engineering: scripting the demo, recording a clean voice-over, and editing out cold-start latency took as long as building the last two features. The corollary is that for a one-semester project, the demo should be

planned and rehearsed inside the build cycle rather than left for the final week, because a demo recording exposes UI gaps that no amount of unit testing surfaces.

A fifth lesson concerned the value of writing down the failure mode of every endpoint before writing the handler itself. The registration endpoint, for example, has four distinct failure modes (duplicate username, duplicate email, malformed payload, transient database error), and each one maps to a different HTTP status code. Listing them on paper before opening the editor cut the implementation time roughly in half, because the validation logic became a sequence of explicit checks against a known matrix rather than a series of ad-hoc try-except blocks added in response to a failing manual test.

A sixth lesson was about the importance of keeping the Postman collection in version control. Treating the collection as code, diffing it alongside the FastAPI routes, and reviewing changes to request payloads in the same pull request as the route changes turned the collection from a one-off exploration tool into a living API specification that doubled as a smoke test. The collection has been exported as a JSON file under repository path `tests/postman_collection.json` and is re-imported into Postman after every pull from main.

6.3 Future Work

Several extensions are natural follow-ups to the present submission. Each item below could be a self-contained sprint of one to three weeks for a single student.

- Add automated unit tests for the Pydantic models, the notes upsert logic and the JSON post-processing functions in the mind-map and flashcard routes.
- Replace the simple proxy with a richer rewriter that strips X-Frame-Options, rewrites relative URLs and applies a content security policy.
- Move from server-stored conversation history to a memory-efficient summary-then-window strategy so long study sessions stay within the model's context budget.
- Add spaced-repetition scheduling on top of the existing flashcards so the user is shown cards they are likely to forget, rather than the whole deck on every open.
- Add collaborative editing for shared study groups, with operational transforms or CRDTs as the synchronisation primitive.
- Mobile-first responsive layouts for the workspace, then a React Native or Flutter wrapper so the same backend powers a phone app.

- Per-project export to a single PDF or printable Word document, so the user can hand their study material to a tutor or print it.
- Optional self-hosted inference using an open-weights model running on the same VM as the FastAPI service, for privacy-sensitive users.

Add a per-user concurrency semaphore around the Groq chatbot endpoint so that bursty user behaviour cannot exhaust the team's tokens-per-minute budget, as observed in Section 5.7.

Migrate the Groq API key from a static environment variable to a rotating secret managed by a dedicated secrets store, in line with the hardening plan in Section 3.8.

Add a lightweight feature flag service so that experimental UI changes (such as a redesigned mind-map view) can be rolled out to a small slice of users without requiring a fresh deployment.

Add a pytest-based unit-test suite covering the JSON parsing in the mindmap and flashcard routes, the page-number invariant in the notes upsert, and the truncation logic in the chatbot prompt builder.

Add a Playwright-based end-to-end smoke job that runs after each Vercel and Render deployment, driving register, login, save-page, and generate-flashcards through a real browser.

Investigate per-project encryption-at-rest of the pages content using a user-derived key, so a Mongo Atlas backup leak would not directly expose study notes.

Build a small CLI companion that imports markdown files from the filesystem into Notify projects, so users with existing notes in Obsidian or Bear can migrate without copy-paste.

A seventh future-work item is to support exporting any project as a self-contained PDF that interleaves the original notes, the cached mind map, and the generated flashcards into a single printable study packet. The skeleton of this feature already exists because the notes are stored as markdown and the mind map is stored as structured JSON; what is missing is a server-side renderer that lays out the three artefacts onto A4 pages with consistent typography and a per-project cover sheet.

An eighth future-work item is to add a lightweight collaboration primitive: a read-only share link that lets the owner generate a URL granting view-only access to a single project for a fixed duration (for example seventy-two hours), with no requirement for the recipient to register a Notify account. This is the minimum viable form of sharing that would let study groups exchange revision material without the implementation complexity of full multi-user editing.

Appendices

Appendix A Sample API Payloads

This appendix lists request and response payloads for the most important endpoints. Each block shows the cURL invocation followed by the JSON body. All examples assume the FastAPI service is reachable at <http://localhost:8000>.

```
# POST /register
curl -X POST http://localhost:8000/register -H "Content-Type: application/json" -d '{
    "username": "tanish",
    "email": "tanish@example.com",
    "password": "correct-horse-battery-staple"
}'

# 200 -> { "message": "User registered successfully" }

# POST /login
curl -X POST http://localhost:8000/login -H 'Content-Type: application/json' -d '{
    "username": "tanish",
    "password": "correct-horse-battery-staple"
}'

# POST /notes/save_note
curl -X POST http://localhost:8000/notes/save_note -H 'Content-Type: application/json' -d '{
    "username": "tanish",
    "project_title": "DBMS Notes",
    "pages": [
        {"note_title": "Normalisation", "page_number": 1,
         "content": "# 1NF, 2NF, 3NF examples ..."}
    ]
}'

# POST /mindmap (refresh forces regeneration)
curl -X POST http://localhost:8000/mindmap -H 'Content-Type: application/json' -d '{
    "username": "tanish",
    "project_title": "DBMS Notes",
    "refresh": false
}'

# POST /flashcards
curl -X POST http://localhost:8000/flashcards -H 'Content-Type: application/json' -d '{
    "username": "tanish",
    "project_title": "DBMS Notes"
}'

# POST /chatbot
curl -X POST http://localhost:8000/chatbot -H 'Content-Type: application/json' -d '{
    "username": "tanish",
    "project_title": "DBMS Notes",
```

```
"message": "Explain BCNF in one paragraph."
}'

# POST /proxy?url=https://en.wikipedia.org/wiki/Database_normalization
```

Appendix B Chatbot System Prompt

The chatbot ships with the following system prompt. It is concatenated with the conversation history and the user's notes before the call to Groq.

You are a helpful assistant whose main goal is to help users with their examination. You are knowledgeable and patient. For each question asked to you, you use your knowledge to provide the best possible answer. You should also ask a related question to keep the conversation going. Correct the user if they ask a question that is not related to the exam. Correct the user if they do not answer the question correctly. You should also provide proper feedback on the user's answer.

Appendix C Repository Layout

The repository is rooted at NOTIFY. The high-level layout is reproduced below for the reviewer.

```
NOTIFY/
README
backend/
  main.py
  database.py
  models.py
  requirements.txt
  routes/
    register.py
    login.py
    get_user.py
    notes.py
    get_notes.py
    mindmap.py
    flash_cards.py
    chatbot.py
    proxy.py
  AI_backend/
    mindmap.py      (standalone mindmap micro-service prototype)
notify/
  package.json
  next.config.mjs
  tailwind.config.ts
  src/
    app/
      page.tsx, layout.tsx, lib.ts, StoreProvider.tsx
      login/, register/, logout/
      Notes/, TinyMCE/, flash_cards/, browserpage/
```

```

editNotes/[projectTitle]/[noteTitle]/[PageNumber]/page.tsx
api/proxy/
components/ (Hero, Tiles, NotesList, NoteCard, SidePanel,
              Browser, FloatingDoc, FloatingComponent,
              MindMap, flashCard, login, signup, UserGreeting)
styles/, fonts/, assets/
features/
  store.ts
  User/user-slice.ts
  Notes/Notes-slice.ts
Weekly_Progress_Reports/
  Week_01_Progress_Report.pdf ... Week_12_Progress_Report.pdf
  All_12_Weekly_Progress_Reports.pdf

```

Appendix D Glossary

Terms used throughout the report are defined here so the reviewer does not have to leave the document to look them up.

- Bcrypt — adaptive password-hashing function used to store passwords in a form that resists brute-force attacks.
- CORS — Cross-Origin Resource Sharing, the browser policy that controls whether one domain can call APIs on another.
- CRUD — Create, Read, Update, Delete; the four basic database operations.
- FastAPI — modern Python web framework with automatic OpenAPI and Pydantic-based request validation.
- GoJS — JavaScript library for interactive diagrams; used here for the mind-map view.
- Groq — hosted-inference platform; runs Llama 3.1 and other open-weights models behind a low-latency API.
- Llama 3.1 — open-weights large language model from Meta; the version used in Notify is llama-3.1-8b-instant on Groq.
- MongoDB Atlas — managed MongoDB cluster used as the production database.
- Motor — asynchronous Python driver for MongoDB, used inside FastAPI.
- Next.js — React framework with file-based routing and built-in server-side rendering; used for the frontend.

- Pydantic — runtime validation library for Python; defines the request and response shapes of the API.
- Redux Toolkit — opinionated Redux distribution that ships slices and selectors out of the box.
- TinyMCE — rich-text editor used as the WYSIWYG note editor.
- Vercel — hosting platform for Next.js applications; used for the production frontend.

Appendix E Evaluation Worksheet

This worksheet is used during the supervisor review to score the project on six dimensions. Each row records the dimension, the evidence used to score it, the maximum score and the self-reported score for the current submission.

Dimension	Evidence	Max	Self
Problem framing	Sec. 1.3, 1.4, 1.5	10	9
Architecture	Sec. 3.1 - 3.7, Fig. 3.1, 3.2	20	18
Implementation completeness	Ch. 4, all nine routes live	25	23
Testing	Sec. 5.2 - 5.6, 16 e2e scenarios	15	13
Deployment	Vercel + Render + Atlas + smoke tests	10	10
Report quality	This document, 40+ pages, IEEE refs	10	9
Innovation / AI integration	Groq Llama 3.1 across 3 features	10	9
	TOTAL	100	91

Appendix F Deployment Checklist

The checklist below was used as the runbook for the Week 12 deployments to Vercel and Render. Each item must be ticked before the deployment is announced.

- MongoDB Atlas cluster is in M0 free tier, IP allow list includes 0.0.0.0/0 only for development and the Render egress IP for production.
- MONGO_SRV is set on Render and points at the Atlas connection string with the correct user, password and database name.
- GROQ_API_KEY is set on Render and is rotated whenever the key is checked into the repository by mistake.
- FastAPI CORS allow_origins is restricted to the Vercel production URL before launch.
- NEXT_PUBLIC_API_FETCH_API on Vercel points at the Render HTTPS URL of the FastAPI service.

- Both services pass the smoke test of register, login, save and flashcards within one minute of deploy.
- Demo video is recorded and uploaded; the README links to it and to the live URLs.

References

- [1] S. Ramirez, "FastAPI: A modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints," FastAPI documentation, 2024. [Online]. Available: <https://fastapi.tiangolo.com>.
- [2] Vercel Inc., "Next.js Documentation," 2024. [Online]. Available: <https://nextjs.org/docs>.
- [3] MongoDB Inc., "MongoDB Manual: Modeling Embedded vs Referenced Documents," 2024. [Online]. Available: <https://www.mongodb.com/docs/manual>.
- [4] Meta AI, "The Llama 3 Herd of Models," arXiv:2407.21783, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>.
- [5] Groq Inc., "Groq Cloud API Reference," 2024. [Online]. Available: <https://console.groq.com/docs>.
- [6] D. Abramov and the Redux team, "Redux Toolkit," 2024. [Online]. Available: <https://redux-toolkit.js.org>.
- [7] Northwoods Software, "GoJS — Diagrams for JavaScript and TypeScript," GoJS Documentation, 2024. [Online]. Available: <https://gojs.net>.
- [8] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999, pp. 81-91. (Origin of bcrypt.)
- [9] Tailwind Labs, "Tailwind CSS — A utility-first CSS framework," 2024. [Online]. Available: <https://tailwindcss.com>.
- [10] Mongo Engineering, "Motor: Asynchronous Python driver for MongoDB," PyMongo documentation, 2024. [Online]. Available: <https://motor.readthedocs.io>.
- [11] G. van Rossum, B. Warsaw and N. Coghlan, "PEP 8: Style Guide for Python Code," Python Enhancement Proposal, 2001. [Online]. Available: <https://peps.python.org/pep-0008/>.
- [12] TinyMCE, "TinyMCE — The Most Trusted Rich Text Editor," Tiny Documentation, 2024. [Online]. Available: <https://www.tiny.cloud/docs>.
- [13] H. Wickham, "Tidy Data," Journal of Statistical Software, vol. 59, no. 10, pp. 1-23, 2014. (Cited for the data-shape philosophy used in the notes schema.)
- [14] S. Allamaraju, RESTful Web Services Cookbook. O'Reilly, 2010.
- [15] M. Nygard, Release It! Design and Deploy Production-Ready Software, 2nd ed. Pragmatic Bookshelf, 2018. (Used as a reference for the deployment checklist in Appendix F.)

Personal Details

- Name: Tanish Bhole
- Enrolment Number: 221B407
- Programme: B.Tech in Computer Science & Engineering
- Semester: VIII
- Project Number: PSB8
- Project Title: Notify - A Virtual Learning Companion
- Supervisor: Dr. P. S. Banerjee, Assistant Professor
- Institute: Jaypee University of Engineering & Technology, Guna (M.P.) - 473226
- Academic Year: 2025 - 2026
- Email: 221B407@juetguna.in
- Repository: github.com/TanishBhole0911/NOTIFY